

ledgerline

A SaaS subscription billing engine. Proration as a ledger, not a formula.
Plans, metered usage, webhook-driven state sync and dunning.

Project report: what it does, how it was tested, and how to run it in production.

97

TESTS (94 pass, 3 need real PG)

7 / 7

DEFINITION OF DONE

6 / 6

HARD REQUIREMENTS

~6,700

LINES (src + tests)

Run it in one command. `npm install && npm test` passes with no Docker, no Postgres install, no Redis and no Stripe key. The database is real Postgres 16 (PGLite, compiled to WASM and running in-process), so CHECK constraints, ON CONFLICT, transaction rollback and FOR UPDATE SKIP LOCKED behave exactly as they will in production. Set DATABASE_URL, REDIS_URL and STRIPE_SECRET_KEY to switch to the real stack without touching a line of code.

1. What it does

A subscription billing system built on one organising principle: **this engine owns the money math, and Stripe owns whether money moved.** Stripe is authoritative about payment outcomes, which arrive by webhook and are the only thing permitted to write a paid or failed status. Everything else - proration, usage aggregation, invoice assembly - is computed and verified locally, because those are the numbers a customer will dispute and you have to be able to defend them line by line.

Capabilities

AREA	WHAT IT DOES
Plans	Tiered plans with a base price plus an optional metered component (for example \$29/mo + \$0.01 per API call over 10,000). Monthly, yearly, weekly or daily intervals. Optional free trials.
Subscriptions	Subscribe, upgrade, downgrade, cancel immediately or at period end. State machine: trialing -> active -> past_due -> canceled, plus paused. Base price billed in advance, metered usage billed in arrears.
Proration	Mid-cycle plan changes credit unused time and charge for remaining time, computed against what was <i>actually billed</i> rather than list price. Handles repeated changes within one cycle.
Metered usage	Usage events recorded in real time with client idempotency keys, aggregated per billing period, correct at the cycle boundary and for events that arrive after their period closed.
Webhooks	Signature-verified ingestion with exactly-once processing and tolerance for out-of-order delivery. Local subscription state cannot drift from Stripe.
Dunning	Failed payments move to past_due and retry on a configurable schedule (days 1, 3, 7 by default), then auto-cancel. Customer notified at every stage.
Billing history	Every invoice - paid or failed - stored with full line-item detail and viewable per customer.
Admin	Live dashboard: MRR, revenue by plan, failed-payment rate, subscription status breakdown, dunning queue, webhook processing health.

Architecture

The layering exists to make the hard parts testable. Everything in `domain/` is pure - no database, no clock, no network - so proration and the dunning schedule are tested as arithmetic rather than through a running system.

LAYER	FILES	LINES	RESPONSIBILITY
<code>src/domain</code>	7	830	Pure billing rules: proration ledger, usage aggregation, money arithmetic, period math, event ordering, dunning schedule, state machine. No I/O.
<code>src/services</code>	8	1,861	Transactional orchestration: subscriptions, invoices, usage, webhooks, dunning, payments, admin analytics, notifications.
<code>src/stripe</code>	3	424	Signature verification, live/offline client selection, in-memory Stripe double.
<code>src/jobs</code>	3	199	Dunning and renewal workers behind a scheduler that is either BullMQ or an in-process Postgres poller.
<code>src/http</code>	7	445	Express routes. The webhook route is mounted raw and first.
<code>src/db</code>	2	376	<code>schema.sql</code> and an adapter that speaks both node-pg and PGLite.
<code>test</code>	8	2,162	97 tests. Assertions are hand-calculated literals, not re-derived from the code under test.

The money path, end to end

- | | |
|---------------------------|---|
| 1. POST /subscriptions | ledger entry + invoice written in ONE transaction |
| 2. COMMIT | -- transaction closed before any network call |
| 3. Stripe invoices.pay() | no DB transaction open (never hold locks over HTTP) |
| 4. POST /webhooks/stripe | signature verified against RAW bytes |
| 5. INSERT webhook_events | + side effects, again in ONE transaction |
| 6. status = paid failed | the webhook is the ONLY writer of payment status |

A 200 from Stripe never marks an invoice paid. That is how local state drifts.

2. The four hard problems

The brief identified correctness under real-world messiness as the difficulty. These are the four places where the obvious implementation is wrong.

2.1 Proration is a ledger, not a formula

The obvious implementation prorates the plan's *list price*. For a chain of plain list-price changes that is algebraically identical to what this engine computes, and the test suite asserts exactly that rather than overstating the case. It breaks the moment the amount actually billed is not the list price prorated - a coupon, a mid-cycle price change, a partial-period signup:

Customer on a \$29 plan with a 50% coupon is billed 1450, upgrades on day 10 of 30:

```
naive credit = 2900 * 20/30 = 1933  <-- against a payment of only 1450
ledger credit = 1450 * 20/30 = 967
```

The naive path refunds 966 cents that were never collected, silently.

So `billed_items` records the amount **actually charged** for each span, and a plan change credits the unused fraction of that, over that item's own span. The credit denominator is the item's span; the charge denominator is the full period. Those two denominators differing is the whole point. The invariant - never credit back more than you charged - is enforced both in code and by a CHECK (`amount_cents >= 0`) constraint, so it holds even if a future code path forgets the check.

Money is integer cents throughout. `mulDivRound` does exact $(a*b)/c$ in `BigInt` because a high-volume metered line reaches 10^{15} , uncomfortably close to the float safe-integer limit. Rounding is half-away-from-zero so credit and charge pairs stay symmetric.

2.2 Metered usage: two different boundary problems

These get conflated and they need different fixes. **An event at the exact rollover instant** is handled by making every period half-open, `[start, end)` - the boundary instant belongs to the next period, counted once, never twice, never dropped. **An event that belongs to a period but arrives after it was invoiced** is a different problem: a request finishing at 23:59:59 can reach the meter seconds later, and you can neither edit a finalised invoice nor drop the usage.

The fix is to aggregate by **claim** rather than by time window. Each event carries `billed_invoice_id`, NULL until an invoice claims it. Invoicing selects unclaimed events with `ts < periodEnd` and no lower bound, so stragglers from a closed period sweep onto the next invoice automatically. Ordinary and late events flow through one code path, so there is no separate branch to forget.

```
UPDATE usage_events SET billed_invoice_id = $1
WHERE subscription_id = $2 AND billed_invoice_id IS NULL AND ts < $3
RETURNING quantity;      -- claims AND counts in a single atomic statement

SELECT sum() then UPDATE would let a concurrent insert be counted but not claimed
(billed twice) or claimed but not counted (billed never).
```

2.3 Webhooks: idempotency and ordering

Idempotency. The insert into `webhook_events` and every side effect share **one transaction**. A duplicate hits the `UNIQUE` index on `stripe_event_id` and returns early with nothing applied; a crash rolls back *including the ledger row*, so Stripe's redelivery is seen as new. Recording first and processing afterwards - the obvious approach - gets this exactly backwards: a crash between the two leaves a row saying "seen" for an event whose effects never happened, and the retry is then discarded as a duplicate. The event is lost permanently and silently. There is a test for precisely this.

Ordering. Two mechanisms, because there are two problems. A *per-invoice ordering guard* stores the order key of the last event applied; a strictly older event is recorded as stale and applied to nothing. The key is [`created`, `attempt_count`, `typeRank`] - `created` alone is not enough because Stripe timestamps have one-second granularity, so a failure and its retry genuinely collide. `attempt_count` totally orders a dunning sequence regardless of clock resolution; that is real ordering information, not a heuristic.

Separately, *subscription status is derived* - recomputed from the current set of invoice facts after each event, never mutated event-by-event. A per-subscription timestamp guard is subtly wrong: invoice A succeeding at `t=200` would make a genuine failure of invoice B at `t=150` look stale, leaving the subscription active while an invoice is unpaid. Deriving status from facts makes arrival order irrelevant, and the suite asserts that replaying the same events in either order converges on the same state. canceled is terminal, so a late success cannot resurrect a subscription dunning already gave up on.

Signatures are verified against the raw request bytes, in constant time, within a replay window. The webhook route is mounted with `express.raw()` *before* `express.json()`; if JSON parsing ran first the original bytes would be gone, every signature would fail, and the usual "fix" is to weaken the check. That ordering is a security property and a test would catch its regression.

2.4 Dunning

Retries are scheduled as **day offsets from the first failure**, not as gaps between attempts. Offsets-from-origin means a delayed worker cannot make the schedule drift; with gap-based scheduling one slow run stretches every subsequent retry. One row per (invoice, attempt) with a `UNIQUE` index makes scheduling idempotent, so a redelivered failure or a restarted worker cannot double the retry queue and charge the customer twice. Workers claim with `FOR UPDATE SKIP LOCKED`.

Attempts live in Postgres even when BullMQ drives the timers: Redis holds *when*, Postgres holds *whether*. Flushing Redis costs latency, never correctness.

3. Requirements fulfilled

3.1 Definition of Done

#	REQUIREMENT	STATUS	EVIDENCE
1	Proration math verified correct against manually calculated test cases	MET	Expected values hand-computed in comments, asserted as literals. The brief's case (\$29 -> \$99, day 10 of 30) yields credit -\$19.33, charge \$66.00, net \$46.67.
2	Metered usage aggregates correctly, including cycle-boundary events	MET	Boundary event lands in the next period; a late arrival timestamped inside a closed period is swept onto the following invoice, not dropped.
3	Duplicate webhook delivery verified to not double-process	MET	Second delivery returns 'duplicate'; one receipt sent, usage not re-counted.
4	Out-of-order webhook delivery handled without corrupting state	MET	Success (t=200) then stale failure (t=100) leaves the subscription active and schedules no dunning. Replay in either order converges.
5	Dunning retries fire on schedule, auto-cancel after configured attempts	MET	Full cycle test: retries at days 1, 3, 7, then canceled, with notifications at each stage.
6	Invoice history accurate and viewable	MET	Verified live: a customer with a declined payment returns both the failed and the paid invoice with full line detail.
7	Admin dashboard shows correct revenue/status breakdown	MET	Verified live against seeded data: revenue by plan, 16.7% failed rate (1 of 6), status counts, MRR \$227.00.

3.2 Hard requirements

#	REQUIREMENT	STATUS	NOTES
1	Plans & subscriptions, state machine	MET	Tiers with base + metered component; subscribe/upgrade/downgrade/cancel; trialing, active, past_due, canceled and paused.
2	Proration, including multiple changes per cycle	MET	Ledger design; tested through four changes in one cycle.
3	Metered usage billing	MET	Metered charge calculated locally - the brief permits either this or reporting usage records to Stripe.
4	Webhook-driven state sync	MET	Signature verification, idempotency, timestamp-based ordering.
5	Dunning	MET	Configurable schedule and retry count; console notifications when no email key is set, as the brief allows.
6	Invoice & billing history, admin view	MET	Per-customer history plus revenue by plan, failed rate and status counts.

3.3 Stack, and what is not yet exercised

COMPONENT	BRIEF	STATUS
Node.js + Express	Required	Implemented and exercised
Postgres	Required - real transactions	Implemented and exercised
Stripe (test mode)	Required	Implemented, NOT exercised against a live key
BullMQ + Redis	Required	Implemented, NOT exercised against real Redis

Being explicit about the gaps. The live Stripe integration is written against the documented API but has never run against a real test key - none was available on the build machine. The BullMQ scheduler is written and the module loads, but no Redis was installed, so only the Postgres-backed poller path has actually run. Three concurrency tests skip unless DATABASE_URL points at a real server, because PGlite is single-connection and cannot produce genuine races. Two minor deviations: the state is spelled canceled (Stripe's convention) rather than the brief's "cancelled", and there is no HTTP endpoint to pause a subscription - the state exists in the machine and syncs from Stripe, but the brief listed paused as optional.

4. The test suite

97 tests across 8 files. The design rule throughout: **expected values are hand-calculated and asserted as literals**. An assertion that re-derives its expectation using the same code it is testing proves nothing. Where a number comes from compounded rounding, the derivation is written out in a comment above the assertion.

FILE	TESTS	COVERS
proration.test.js	15	The brief's checkpoint case; upgrades, downgrades, changes at exact period start and end; two and four changes in one cycle; credits against billed vs list price; seconds-not-days precision; 31-day months; rounding; over-credit detection.
webhooks.test.js	19	Valid/tampered/wrong-secret/missing/malformed signatures; replay window; secret rotation; duplicate delivery; crash rollback and reprocessing; reversed delivery; same-second ties by attempt_count and by type rank; terminal cancellation; order convergence.
domain.test.js	23	BigInt-exact money arithmetic; rounding symmetry; month-end anchor clamping; leap years; half-open containment; the full state-transition matrix; Stripe status mapping; event order keys.
usage.test.js	15	Half-open boundaries; late arrivals; claim-exactly-once; client idempotency keys; sub-cent rates; 10 ⁹ -unit precision; projection matching the eventual invoice.
dunning.test.js	11	Schedule from origin not gaps; configurability; validation; past_due transition; idempotent scheduling under a new event id; full retry-to-cancellation cycle; recovery on successful retry.
integration.test.js	11	Cash reconciliation across two plan changes; trial conversion; billing anchor across a short month; cancellation semantics; admin dashboard; HTTP surface including raw-body signature verification and duplicate acknowledgement.
concurrency.test.js	3	Skipped without a real Postgres. Concurrent duplicate deliveries; concurrent plan changes crediting one ledger item; two workers claiming one dunning attempt.

Two real bugs the tests caught

Downgrade credit granted twice. The proration invoice recorded a negative total *and* banked the same credit, which the next invoice then applied again. Found by reconciling total cash collected against the ledger: \$75.73 collected against \$100.57 owed. Fixed with an explicit balancing line so a downgrade invoice nets to zero and the money moves only via the credit balance. The reconciliation test now pins this down.

Webhook status collision. processwebhook spread the handler's return value over its own result, so a handler returning status: 'canceled' silently overwrote the processing status 'processed' - meaning every caller, including the HTTP response, read the wrong field. Fixed by nesting the handler outcome under detail.

Running them

```
npm test                # 94 pass, 3 skip, no setup required
npm run demo            # narrated walk through all four problems

docker compose up -d    # real Postgres + Redis
DATABASE_URL=postgres://billing:billing@localhost:5432/billing npm test
                        # unskips the 3 concurrency tests
```

5. Using it in real life

5.1 Local, in under a minute

```
npm install
npm run seed            # plans + 4 customers: healthy, upgraded, in dunning, on trial
npm start               # http://localhost:3000/admin
```

With no environment configured this runs on in-process Postgres persisted to `./.data/pglite`, an offline Stripe double, and a Postgres-backed job poller. Notifications print to the console and are persisted to the notifications table. Payments still settle through the real webhook path, signed and verified - offline mode exercises production code rather than a shortcut around it.

5.2 Connecting real Stripe

```
export STRIPE_SECRET_KEY=sk_test_...
stripe listen --forward-to localhost:3000/webhooks/stripe # prints whsec_...
export STRIPE_WEBHOOK_SECRET=whsec_...
npm start

stripe trigger invoice.payment_failed # exercises the dunning path for real
```

Send the same event twice and `/admin/webhooks` should show one processed and one duplicate. That single check confirms signature verification, the raw-body mount and the idempotency ledger are all wired correctly - it is the first thing to run after deploying.

5.3 Production stack

```
DATABASE_URL=postgres://user:pass@host:5432/billing # switches to node-pg + pooling
REDIS_URL=redis://host:6379                        # switches to BullMQ
STRIPE_SECRET_KEY=sk_live_...
STRIPE_WEBHOOK_SECRET=whsec_...                    # from the Dashboard endpoint
RESEND_API_KEY=re_...                               # real email instead of console
DUNNING_RETRY_DAYS=1,3,7,14                         # tune without code changes
DUNNING_MAX_ATTEMPTS=4

node scripts/migrate.js && node src/server.js
```

Nothing else changes. The database adapter, job scheduler, Stripe client and notifier all select their implementation from these variables at boot; `GET /health` reports which of each is active, which is worth asserting in a smoke test so a missing variable cannot silently leave you on the offline double.

5.4 Operating it

CONCERN	WHAT TO DO
Scaling out	Run several instances freely. Dunning claims use FOR UPDATE SKIP LOCKED and webhook idempotency is a UNIQUE index, so correctness comes from the database, not from running a single worker.
Webhook endpoint	Must be publicly reachable and must return 2xx quickly. A 5xx is deliberate back-pressure: the transaction rolled back and Stripe should redeliver. Never make it return 200 on error.
Monitoring	Watch /admin/dashboard: a rising failed-payment rate signals card or processor trouble; a non-zero failed webhook count means events are being rejected and money is going unrecorded. Non-zero duplicate and stale counts are healthy - they are the guards doing their job.
Reconciliation	Run the cash-reconciliation check from the integration test against production data periodically: total invoiced should equal what the ledger says was consumed. That is the check that caught the double-credit bug.
Backups	billed_items is the proration ledger and webhook_events is the idempotency ledger. Losing either means losing the ability to prorate correctly or to reject replays. Back up like financial records, because they are.
Timezones	All billing math is in UTC epoch seconds. Present local times in the UI if you like, but never let a local date reach the billing path.

5.5 Before you take real money

This is a complete and tested billing engine, but a production deployment handling live cards needs the following, none of which the brief asked for:

ITEM	WHY
Authentication and authorisation	Every route is currently open. The admin endpoints expose revenue across all customers, and the customer endpoints expose billing history. This is the single largest gap between the current state and production.
Verify the live Stripe path	The integration is written but unexercised. Run the full lifecycle against a test key before trusting it, and confirm that invoice items land with the amounts this engine computed.
Verify BullMQ against real Redis	The poller path is proven; the BullMQ path is not. Confirm repeatable jobs fire and that flushing Redis does not lose scheduled dunning.
Rate-limit the usage endpoint	It accepts high-frequency writes by design. Without limits it is the easiest way to overwhelm the database.
Tax and multi-currency	Explicitly out of scope per the brief. Currency is stored per-invoice but no conversion or tax logic exists. Do not add tax by adjusting line amounts - it needs its own lines.
Structured logging and alerting	Currently console. Route webhook failures and dunning cancellations to real alerting - an auto-cancellation is a customer about to churn.

5.6 API surface

GET	/health	runtime: which DB, queue, Stripe and email are live
GET	/plans	POST /plans
POST	/customers	GET /customers/:id
GET	/customers/:id/invoices	full billing history with line items
GET	/customers/:id/notifications	
POST	/subscriptions	{ customerId, planId, at? }
GET	/subscriptions/:id	
POST	/subscriptions/:id/change-plan	{ planId, at? } upgrade or downgrade
POST	/subscriptions/:id/cancel	{ atPeriodEnd? }
POST	/subscriptions/:id/renew	force a period rollover
POST	/subscriptions/:id/usage	{ quantity, timestamp?, idempotencyKey? }
GET	/subscriptions/:id/usage	projected metered charge for the open period
GET	/invoices	GET /invoices/:id
POST	/webhooks/stripe	POST /invoices/:id/pay signed, raw body, mounted first
GET	/admin/dashboard	revenue, MRR, failed rate, status counts
GET	/admin/webhooks	processing ledger: processed/duplicate/stale
GET	/admin/dunning	the retry queue
POST	/admin/dunning/run	{ at? } run due retries at a simulated time

Every at parameter accepts epoch seconds or an ISO string. They exist so the billing clock can be advanced deliberately - you can demonstrate a full seven-day dunning cycle in a single request rather than waiting a week. In production they are simply omitted and the real clock is used.

Every figure in this report was produced by an actual run: `npm test` for the test counts, live HTTP queries against a seeded server for the dashboard and invoice figures, and line counts from the source tree. Where something was not verified, it is labelled as such.